DÉPLOIEMENT D'APPLICATIONS INTELLIGENTES · MLOps

TP 6 — Créer et déployer un modèle via une plateforme AutoML

Durée : 3 h · Partie 5 du cours · Windows PowerShell

Cycle complet sur une plateforme AutoML, entièrement en local et avec des outils open source : démarrer la plateforme H2O-3 et son interface web H2O Flow, lancer une recherche AutoML reproductible, explorer le classement des modèles, enregistrer le meilleur dans le registre MLflow, le déployer derrière un point de terminaison, l'appeler depuis un client et mesurer sa latence. On refait ensuite la même chaîne avec FLAML, la bibliothèque légère vue au TP 5 et dans la partie 5 du cours, pour comparer ce que chaque outil coûte une fois en production. Aucun compte cloud n'est nécessaire.

VERSION WINDOWS

Toutes les commandes sont écrites pour Windows PowerShell (5.1 ou 7), jamais l'invite cmd.exe. Les blocs `@' ... '@` écrivent un fichier ou l'envoient à Python : le marqueur de fermeture `'@` doit être en tout début de ligne, et le bloc se colle en entier. Les fichiers sont écrits en `-Encoding ascii` : l'encodage UTF8 de PowerShell 5.1 ajoute un marqueur invisible en tête de fichier qui casse la lecture des fichiers YAML et JSON.
Si l'activation de l'environnement est refusée : `Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned`, puis relancez `.venv\Scripts\Activate.ps1`.

CE QUI CHANGE DANS CETTE VERSION

Le parcours Google Vertex AI est remplacé par H2O-3, la plateforme AutoML open source présentée dans la partie 5 du cours : tout se fait sur votre poste, sans compte ni carte bancaire. Toutes les sorties ont été obtenues réellement (H2O-3 3.46.0.12, MLflow 3.17.2, FLAML 2.7.0). Les deux recherches AutoML ont un budget en nombre de modèles ou d'essais, avec une graine : relancées trois fois, elles ont donné exactement les mêmes résultats.

Objectifs pédagogiques

- Démarrer une plateforme AutoML locale et la piloter depuis Python et depuis son interface web
- Lancer une recherche AutoML reproductible et lire son classement (leaderboard)
- Enregistrer, versionner et promouvoir le modèle retenu dans le registre MLflow
- Déployer ce modèle derrière un point de terminaison et mesurer sa latence
- Comparer deux outils AutoML sur la qualité, la latence, la taille et les dépendances
- Identifier ce dont un modèle a besoin pour fonctionner en production

Prérequis

- TP 5 terminé ; serveur MLflow démarré sur le port 5000 (`mlflow server --host 127.0.0.1 --port 5000`)
- **Java 17** (ou 11) : H2O-3 3.46 n'accepte que Java 8 à 17
- Environ 2 Go de mémoire vive libres pour le serveur H2O

Installation

Vérifiez d'abord Java. S'il manque ou si la version est supérieure à 17, installez un JDK 17 (commande `winget` ci-dessous, ou Temurin 17 sur adoptium.net), puis rouvrez PowerShell. Le paquet Python `h2o` contient la plateforme complète (un fichier `h2o.jar` d'environ 280 Mo).

terminal

```
java -version                               # doit afficher 17 (ou 11)
winget install EclipseAdoptium.Temurin.17.JDK   # seulement si Java manque
cd tp-mlops; .venv\Scripts\Activate.ps1
pip install h2o "flaml[automl]" "mlflow>=3.15" requests joblib
```

RÉSULTAT ATTENDU

```
openjdk version "17.0.x"
Successfully installed h2o-3.46.0.12 flaml-2.7.0 lightgbm-4.x ...
```

Déroulement

Étape 1 — Démarrer la plateforme H2O (15 min)

H2O-3 est un serveur Java : il garde en mémoire les données et les modèles, et expose une API et une interface web, **H2O Flow**. Python ne fait que lui envoyer des ordres. On le démarre dans un terminal dédié, qui restera ouvert pendant tout le TP.

terminal 1 : la plateforme

```
cd tp-mlops; .venv\Scripts\Activate.ps1
$JAR = (python -c "import h2o, os; print(os.path.dirname(h2o.__file__))") + "\backend\bin\h2o.jar"
java -jar $JAR -port 54321                  # laisser ce terminal ouvert
```

RÉSULTAT ATTENDU

```
... INFO water.default: Open H2O Flow in your web browser: http://...:54321

Ouvrez http://127.0.0.1:54321 dans votre navigateur : c'est H2O Flow.
```

SI H2O REFUSE DE DÉMARRER

Message réel obtenu avec Java 21 : « Only Java versions 8-17 are supported, system version is 21.0.10 ». Installez Java 17 et vérifiez avec `java -version` que c'est bien lui qui répond. En dépannage seulement, l'option `-allow_unsupported_java` force le démarrage, sans garantie. Si le port 54321 est occupé : `-port 54331`, et adaptez l'adresse dans les scripts.

Étape 2 — Lancer l'AutoML et enregistrer le meilleur modèle (25 min)

Même découpage qu'aux TP 1 à 5. Le budget est fixé en **nombre de modèles** et non en secondes, avec une graine, et l'algorithme Deep Learning est exclu car il n'est pas reproductible par défaut : c'est la leçon de la partie 5 (« deux exécutions, deux scores »). Le meilleur modèle part directement dans le registre MLflow.

terminal 2 : src/h2o_automl.py

```
@'
import h2o, pandas as pd, mlflow, mlflow.h2o
from h2o.automl import H2OAutoML
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score
h2o.init(url='http://127.0.0.1:54321')                      # la plateforme du terminal 1
df = pd.read_csv('data/transactions.csv')
tr, te = train_test_split(df, test_size=0.25, random_state=42)   # decoupage des TP 1 a 5
htr = h2o.H2OFrame(tr, destination_frame='fraude_train')
hte = h2o.H2OFrame(te, destination_frame='fraude_test')
for f in (htr, hte):
    f['fraude'] = f['fraude'].asfactor()                     # classification, pas regression
aml = H2OAutoML(max_models=10, seed=42,                      # budget en modeles : reproductible
                exclude_algos=['DeepLearning'],              # non reproductible par defaut
                sort_metric='AUC')
aml.train(y='fraude', training_frame=htr)
print('projet AutoML :', aml.project_name)          # pour le retrouver dans H2O Flow
lb = aml.leaderboard.as_data_frame()
print(lb[['model_id', 'auc', 'logloss']].head(8).to_string(index=False))
pred = aml.leader.predict(hte).as_data_frame()['predict'].astype(int)
f1 = f1_score(te['fraude'], pred)
print('modele retenu :', aml.leader.model_id, '| F1 test =', round(f1, 4))
mlflow.set_tracking_uri('http://127.0.0.1:5000')
mlflow.set_experiment('tp6-automl-deploiement')
with mlflow.start_run(run_name='h2o-automl-10-modeles'):
    mlflow.log_params({'outil': 'h2o', 'max_models': 10, 'modele': aml.leader.model_id})
    mlflow.log_metric('f1', f1)
    mlflow.h2o.log_model(aml.leader, name='model', input_example=te.drop(columns='fraude').head(3),
```

```
                        registered_model_name='fraude-h2o')
'@ | Set-Content -Encoding ascii src/h2o_automl.py
python src/h2o_automl.py
```

RÉSULTAT ATTENDU

```
projet AutoML : AutoML_1_20261024_124104
                                             model_id      auc  logloss
   StackedEnsemble_AllModels_1_AutoML_1_20261024_124104 0.986723 0.093158
StackedEnsemble_BestOfFamily_1_AutoML_1_20261024_124104 0.986404 0.093773
                     XGBoost_2_AutoML_1_20261024_124104 0.985935 0.095959
                     XGBoost_3_AutoML_1_20261024_124104 0.985751 0.097927
                         GBM_1_AutoML_1_20261024_124104 0.985400 0.100054
                     XGBoost_1_AutoML_1_20261024_124104 0.984459 0.100987
                         GBM_2_AutoML_1_20261024_124104 0.983293 0.107500
                         GBM_3_AutoML_1_20261024_124104 0.981224 0.113844
modele retenu : StackedEnsemble_AllModels_1_AutoML_1_... | F1 test = 0.8464
Successfully registered model 'fraude-h2o'.
Created version '1' of model 'fraude-h2o'.

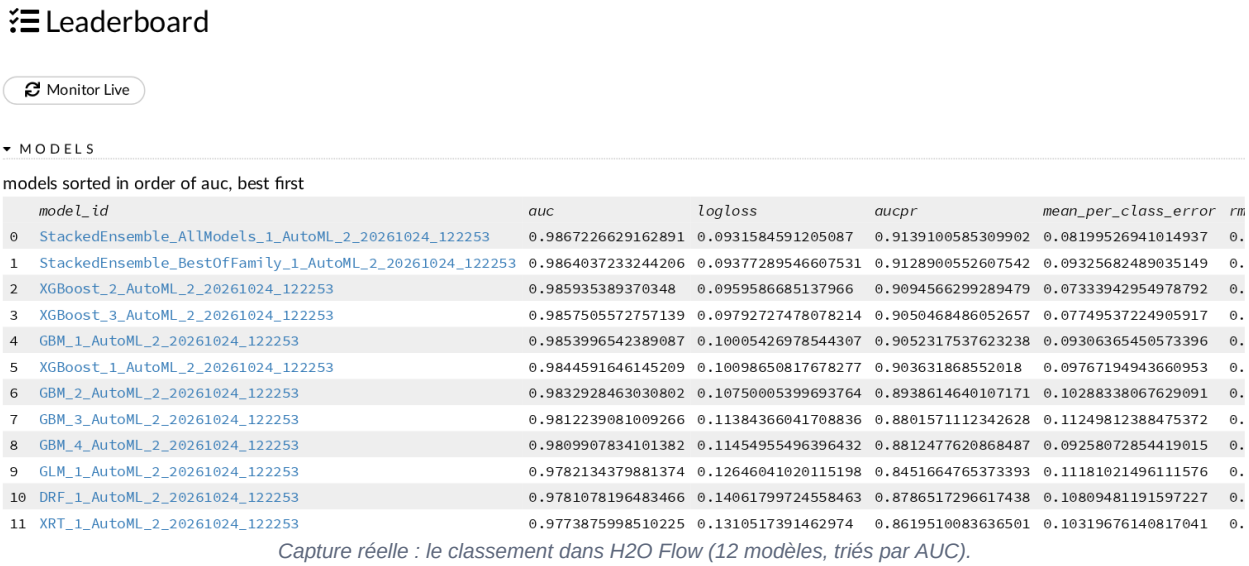
Environ une minute. La date et l'heure dans les noms changent ; les scores, non.
```

Si ça coince : le bandeau « You are running the community edition of H2O-3 OSS » est normal : il rappelle qu'une édition commerciale existe. Les avertissements « single-thread » et « Integer columns » sont sans gravité. Si vous relancez le script, l'AutoML suivant s'appelle `AutoML_2_...` : c'est un nouveau projet dans le même serveur.

Étape 3 — Explorer les résultats dans H2O Flow (20 min)

Partie sans code : on lit dans l'interface web ce que la recherche a produit. H2O Flow fonctionne comme un carnet : on tape une commande dans une cellule et on l'exécute avec **Ctrl+Entrée**, ou on passe par les menus.

1. Dans la première cellule de H2O Flow, tapez `getLeaderboard "AutoML_1_xxx@@fraude"` en remplaçant `AutoML_1_xxx` par le nom affiché par le script (ligne « projet AutoML »), puis **Ctrl+Entrée**. Le classement complet s'affiche : 12 modèles (10 modèles de base + 2 ensembles), avec leurs métriques en validation croisée, et le journal des décisions de l'AutoML (EVENT LOG).
2. Faites défiler l'EVENT LOG : relevez la méthode de validation choisie (« 5-fold cross-validation ») et la graine (« Build control seed: 42 »).
3. Cliquez sur le nom du modèle **XGBoost_2_...** (le meilleur modèle simple) : sa fiche s'ouvre. Repérez la courbe ROC, puis la section **VARIABLE IMPORTANCES** et relevez l'ordre des variables.
4. Toujours dans la fiche, comparez la matrice de confusion **TRAINING METRICS** et celle de **CROSS VALIDATION METRICS** : laquelle est la plus optimiste, et pourquoi ?
5. Menu **Model > List All Models** : vous retrouvez tous les modèles du serveur. Menu **Model > Run AutoML...** : c'est le même lancement que le script, entièrement à la souris (ne le lancez pas, observez seulement les réglages proposés).



Leaderboard

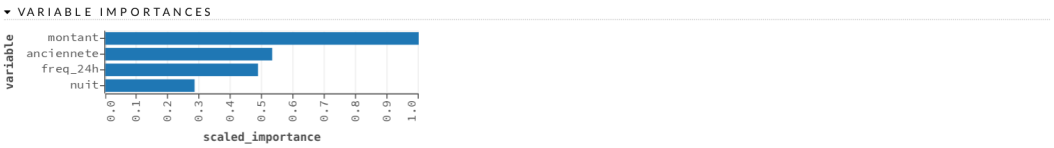
Monitor Live

MODELS

models sorted in order of auc, best first

	model_id	auc	logloss	aucpr	mean_per_class_error	rm
0	StackedEnsemble_AllModels_1_AutoML_2_20261024_122253	0.9867226629162891	0.0931584591205087	0.9139100585309902	0.08199526941014937	0.
1	StackedEnsemble_BestOfFamily_1_AutoML_2_20261024_122253	0.9864037233244206	0.09377289546607531	0.9128900552607542	0.09325682489035149	0.
2	XGBoost_2_AutoML_2_20261024_122253	0.985935389370348	0.0959586685137966	0.9094566299289479	0.07333942954978792	0.
3	XGBoost_3_AutoML_2_20261024_122253	0.9857505572757139	0.09792727478078214	0.9050468486052657	0.07749537224905917	0.
4	GBM_1_AutoML_2_20261024_122253	0.9853996542389087	0.10005426978544307	0.9052317537623238	0.09306365450573396	0.
5	XGBoost_1_AutoML_2_20261024_122253	0.9844591646145209	0.10098650817678277	0.903631868552018	0.09767194943660953	0.
6	GBM_2_AutoML_2_20261024_122253	0.9832928463030802	0.10750005399693764	0.8938614640107171	0.10288338067629091	0.
7	GBM_3_AutoML_2_20261024_122253	0.9812239081009266	0.11384366041708836	0.8801571112342628	0.11249812388475372	0.
8	GBM_4_AutoML_2_20261024_122253	0.9809907834101382	0.11454955496396432	0.8812477620868487	0.09258072854419015	0.
9	GLM_1_AutoML_2_20261024_122253	0.9782134379881374	0.12646041020115198	0.8451664765373393	0.11181021496111576	0.
10	DRF_1_AutoML_2_20261024_122253	0.9781078196483466	0.14061799724558463	0.8786517296617438	0.10809481191597227	0.
11	XRT_1_AutoML_2_20261024_122253	0.9773875998510225	0.1310517391462974	0.8619510083636501	0.10319676140817041	0.

Capture réelle : le classement dans H2O Flow (12 modèles, triés par AUC).



TRAINING METRICS - CONFUSION MATRIX ROW LABELS: ACTUAL CLASS; COLUMN LABELS: PREDICTED CLASS

	0	1	Error	Rate	Precision
0	3280	36	0.0109	36 / 3,316	0.99
1	43	391	0.0991	43 / 434	0.92
Total	3323	427	0.0211	79 / 3,750	
Recall	0.99	0.90			

Capture réelle : importance des variables du modèle XGBoost_2 et sa matrice d'apprentissage.

RÉSULTAT ATTENDU

```
Ordre des variables : montant, anciennete, freq_24h, nuit
Matrice TRAINING de XGBoost_2 : 391 fraudes reconnues sur 434, 36 fausses alertes
sur 3 316 : bien meilleure que la validation croisée, car le modèle a vu ces lignes.
```

LE MEILLEUR MODÈLE EST UN ENSEMBLE

`StackedEnsemble_AllModels` combine les prédictions de tous les autres modèles avec un méta-modèle. Il gagne de quelques millièmes d'AUC, mais il est plus lourd à servir et plus difficile à expliquer : pour justifier une décision, il faut remonter dans dix modèles. Gardez cette remarque pour la question 1.

Étape 4 — Promouvoir et déployer le point de terminaison (15 min)

Comme au TP 3 : un alias `production` désigne la version à servir, et MLflow expose le modèle derrière une API HTTP. Le modèle H2O n'est pas autonome : au chargement, MLflow se connecte au serveur H2O du terminal 1 et y recharge le modèle.

terminal 2

```
@'
import mlflow
from mlflow import MlflowClient
mlflow.set_tracking_uri('http://127.0.0.1:5000')
c = MlflowClient()
v = max(int(m.version) for m in c.search_model_versions("name='fraude-h2o'"))
c.set_registered_model_alias('fraude-h2o', 'production', v)   # derniere version
print('version', v, 'promue')
'@ | python -
```

RÉSULTAT ATTENDU

```
version 1 promue
```

terminal 3 : le point de terminaison H2O

```
cd tp-mlops; .venv\Scripts\Activate.ps1
$env:MLFLOW_TRACKING_URI = "http://127.0.0.1:5000"
mlflow models serve -m 'models:/fraude-h2o@production' `
  -p 8100 --env-manager local
```

RÉSULTAT ATTENDU

```
Checking whether there is an H2O instance running at http://localhost:54321. connected.
...
INFO:     Uvicorn running on http://127.0.0.1:8100
```

Si ça coince : si l'erreur « No module named 'mlflow' » apparaît, c'est que l'environnement n'est pas activé dans ce terminal : la commande `mlflow` lance un serveur `uvicorn` qui doit être celui de `.venv`.

Étape 5 — Consommer le point de terminaison et mesurer la latence (15 min)

Un client externe envoie une transaction suspecte (8 500 MAD, client récent, nuit), puis mesure 50 appels. Le port est passé en argument : on réutilisera le même client à l'étape 6.

terminal 2 : src/client.py

```
@'
import sys, time, statistics, requests
PORT = sys.argv[1] if len(sys.argv) > 1 else '8100'   # 8100 : H2O, 8101 : FLAML
URL = f'http://127.0.0.1:{PORT}/invocations'
CORPS = {'dataframe_records': [{'montant': 8500.0, 'anciennete': 3,
                                'freq_24h': 9, 'nuit': 1}]}
r = requests.post(URL, json=CORPS, timeout=10)
print('reponse :', r.status_code, r.json())
lat = []
for _ in range(50):                            # 50 appels pour mesurer
    t = time.perf_counter()
    requests.post(URL, json=CORPS, timeout=10)
    lat.append((time.perf_counter() - t) * 1000)
lat.sort()
print('mediane', round(statistics.median(lat), 1), 'ms')
print('p95    ', round(lat[int(0.95 * len(lat))], 1), 'ms')
'@ | Set-Content -Encoding ascii src/client.py
python src/client.py 8100
```

RÉSULTAT ATTENDU

```
reponse : 200 {'predictions': [{'predict': 1, 'p0': 9.54e-07, 'p1': 0.999999}]}
mediane 260 a 450 ms   (trois series de mesures)
p95     environ 470 ms

Le modele H2O renvoie la classe ET les deux probabilites.
```

Si ça coince : les latences dépendent du poste et de sa charge ; relancez deux ou trois fois et notez la plage obtenue. Ce qui compte, c'est l'ordre de grandeur, à comparer à l'étape 6.

Étape 6 — La même chaîne avec FLAML (25 min)

Deuxième AutoML, sans serveur à démarrer : FLAML est une simple bibliothèque Python. Même découpage, budget de 25 essais avec une graine. On l'enregistre, on le promeut et on le sert sur le port 8101.

terminal 2 : src/flaml_automl.py

```
@'
import pandas as pd, joblib, mlflow, mlflow.sklearn
from flaml import AutoML
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score
df = pd.read_csv('data/transactions.csv')
X, y = df.drop(columns='fraude'), df['fraude']
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=42)
a = AutoML()
a.fit(Xtr, ytr, task='classification', metric='f1', time_budget=-1,
      max_iter=25, seed=42, verbose=0)              # budget en essais : reproductible
f1 = f1_score(yte, a.predict(Xte))
print('modele retenu :', a.best_estimator, '| F1 test =', round(f1, 4))
modele = a.model.estimator                 # le modele sous-jacent (LightGBM, XGBoost...)
joblib.dump(modele, 'models/model_flaml.pkl')
mlflow.set_tracking_uri('http://127.0.0.1:5000')
mlflow.set_experiment('tp6-automl-deploiement')
with mlflow.start_run(run_name='flaml-25-essais'):
    mlflow.log_params({'outil': 'flaml', 'max_iter': 25, 'modele': a.best_estimator})
    mlflow.log_metric('f1', f1)
    mlflow.sklearn.log_model(modele, name='model', input_example=Xte[:3],
                             serialization_format='cloudpickle',   # voir encadre
                             registered_model_name='fraude-flaml')
'@ | Set-Content -Encoding ascii src/flaml_automl.py
python src/flaml_automl.py
```

RÉSULTAT ATTENDU

```
modele retenu : lgbm | F1 test = 0.8359
Successfully registered model 'fraude-flaml'.
Created version '1' of model 'fraude-flaml'.

Une dizaine de secondes. FLAML n'essaie que les bibliotheques installees : si
catboost est present sur votre poste, le modele retenu peut etre different.
```

POURQUOI serialization_format='cloudpickle' ?

Depuis MLflow 3.16, le format par défaut est skops, un format sécurisé qui ne sait pas enregistrer un modèle LightGBM : l'enregistrement échoue avec « untrusted types ». Le format cloudpickle fonctionne pour tout modèle Python, au prix d'un avertissement : un fichier pickle peut exécuter du code à son chargement, on ne l'accepte donc que de source sûre.

terminal 2

```
@'
import mlflow
from mlflow import MlflowClient
mlflow.set_tracking_uri('http://127.0.0.1:5000')
c = MlflowClient()
v = max(int(m.version) for m in c.search_model_versions("name='fraude-flaml'"))
c.set_registered_model_alias('fraude-flaml', 'production', v)   # derniere version
print('version', v, 'promue')
'@ | python -
```

terminal 4 : le point de terminaison FLAML

```
cd tp-mlops; .venv\Scripts\Activate.ps1
$env:MLFLOW_TRACKING_URI = "http://127.0.0.1:5000"
mlflow models serve -m 'models:/fraude-flaml@production' `
  -p 8101 --env-manager local
```

terminal 2

```
python src/client.py 8101
```

RÉSULTAT ATTENDU

```
version 1 promue
reponse : 200 {'predictions': [1]}
mediane 6 a 8 ms
p95     8 a 10 ms

Meme decision, environ 50 fois plus vite que le modele H2O.
```

Étape 7 — Panne contrôlée : de quoi dépend le service ? (10 min)

Un service en production doit survivre aux redémarrages. Testons ce qui se passe quand la plateforme H2O redémarre alors que le point de terminaison tourne toujours.

1. Dans le terminal 1, arrêtez H2O avec **Ctrl+C**, puis relancez la même commande `java -jar ...`.
2. Dans le terminal 2, relancez `python src/client.py 8100`. Le point de terminaison répond, mais par une erreur : notez-la.
3. Arrêtez le terminal 3 (**Ctrl+C**) et relancez `mlflow models serve` : le modèle est rechargé dans le nouveau serveur H2O, le client répond de nouveau.
4. Faites la même expérience avec FLAML : arrêtez H2O, puis appelez le port 8101. Que constatez-vous ?

RÉSULTAT ATTENDU

```
Erreur reelle renvoyee par le port 8100 (code 400) :
  H2OKeyNotFoundArgumentException:
  Error: Object 'StackedEnsemble_AllModels_1_AutoML_1_...' not found
  in function: predict for argument: model

Le modele vivait dans la memoire du serveur H2O : au redemarrage, il a disparu.
Le service FLAML (port 8101), lui, n'est pas concerne.
```

Étape 8 — Comparer et conclure (20 min)

La comparaison doit porter sur plusieurs axes, pas seulement sur le F1. Complétez le tableau avec vos mesures. Pour la taille : le dossier du modèle dans `mlartifacts/` (ou l'onglet Artifacts de l'interface MLflow).

reports/tp6_comparaison.md

```
@'
| Critere              | Manuel (TP 3) | H2O AutoML       | FLAML         |
|----------------------|---------------|------------------|---------------|
| F1 sur le jeu test   | 0.8669        | 0.8464           | 0.8359        |
| Modele retenu        | foret 300     | Stacked Ensemble | LightGBM      |
| Latence mediane      | ...           | ... ms           | ... ms        |
| Taille du modele     | 7.8 Mo        | ...              | ...           |
| Dependances en prod  | Python        | Python + Java    | Python        |
|                      |               | + serveur H2O    |               |
| Explicabilite        | ...           | ...              | ...           |
'@ | Set-Content -Encoding ascii reports/tp6_comparaison.md
```

RÉSULTAT ATTENDU

```
Valeurs relevees lors de la preparation :
  modele H2O (dossier MLflow)          4,7 Mo
  modele FLAML (models/model_flaml.pkl)   44 Ko
  foret du TP 3 (models/model.pkl)      7,8 Mo
```

Nettoyage en fin de séance : **Ctrl+C** dans les terminaux 1, 3 et 4. Les modèles restent dans le registre MLflow.

OPEN SOURCE NE VEUT PAS DIRE SANS DÉPENDANCE

Le modèle H2O est enregistré dans un format binaire lisible seulement par la même version d'H2O. L'export autonome (MOJO) a été refusé par la version open source testée pour la partie 5 (« requires H2O-3 Secure », l'édition commerciale). Pour pouvoir reconstruire ce modèle dans deux ans, il faut conserver le script, les données versionnées (DVC), la graine et la version exacte d'H2O et de Java.

Questions du compte rendu

1. Le modèle retenu par H2O est un Stacked Ensemble. Qu'est-ce que c'est, et qu'est-ce que cela change pour expliquer une décision à un client dont la carte a été bloquée ?
2. Pourquoi avoir fixé `max_models`, `seed` et exclu Deep Learning ? Qu'aurait-on perdu avec `max_runtime_secs=60` (voir la slide 10 de la partie 5) ?
3. La latence du modèle H2O est environ 50 fois celle du modèle FLAML pour la même décision. D'où vient cet écart ? Quel modèle choisiriez-vous pour autoriser un paiement en temps réel ?
4. À l'étape 7, que révèle le message « Object ... not found » sur l'architecture du service H2O ? Comment détecteriez-vous automatiquement cette panne (TP 7) ?
5. Le réglage manuel du TP 3 a le meilleur F1. Quel modèle déploieriez-vous en production, et sur quels critères autres que le F1 ?
6. Que faut-il conserver pour pouvoir reconstruire à l'identique le modèle H2O dans deux ans ?

Livrables

- Les scripts `src/h2o_automl.py`, `src/flaml_automl.py` et `src/client.py`
- Captures de H2O Flow : classement, fiche d'un modèle avec l'importance des variables
- Capture du registre MLflow montrant `fraude-h2o` et `fraude-flaml` avec l'alias production
- Les sorties du client sur les deux ports, et l'erreur obtenue à l'étape 7
- Le tableau `reports/tp6_comparaison.md` complété et commenté

Barème

Critère	Points
Plateforme démarrée, AutoML reproductible journalisé dans MLflow	4
Exploration dans H2O Flow et lecture critique des résultats	3
Modèle enregistré, promu et servi derrière un point de terminaison	4
Client, mesures de latence et deuxième chaîne avec FLAML	5
Panne contrôlée et comparaison argumentée	4
Total	20